

Article

Research and Implementation of Local Spatiotemporal Event Quantities Denoising Algorithm Based on Event-Based Vision Sensors

Dongchao Gong ¹, Xinting Jia ^{1,*} and Yingping Yang ^{1,2,*}¹ School of Physics and Mechanics, Wuhan University of Technology, Wuhan 430070, China² Department of Basic Science, Wuchang Shouyi University, Wuhan 430063, China

* Correspondence: thomasjxt@whut.edu.cn (X.J.); ypyang@whut.edu.cn (Y.Y.)

Abstract

Event-based vision sensors (EVs), with the core advantages of low latency, high dynamic range, and low data volume, have become one of the research hotspots in the field of computer vision. However, the characteristic of detecting changes in light intensity makes EVs particularly sensitive to noise, so the large number of noise events in the event stream significantly limits the practical application of EVs. To address this critical issue, considering the types and characteristics of noise, this paper proposes an event stream denoising algorithm based on local spatiotemporal event quantities and implements it in hardware. To comprehensively evaluate the algorithm's performance, two metrics based on the probability of real events, Noise Event Ratio (NER) and Event Noise Ratio (ENR), are used to quantify the denoising effect, while hardware resource overhead is assessed in terms of event processing latency and memory usage. Experimental results show that the proposed algorithm achieves an NER of 8.37% and an ENR of 25.10%. Compared to existing denoising algorithms, such as the DWF algorithm, the NER and ENR of this algorithm are reduced by 27.72% and 22.89%, respectively, demonstrating superior denoising performance. On the hardware side, the latency for processing a single event is approximately 110 ns, with a total resource usage of N^2 memory units. Although the hardware consumption is slightly higher, the algorithm exhibits significant advantages in denoising performance, providing effective support for the engineering application of EVs.

Keywords: event-based vision sensors; event streams; noise events; local spatiotemporal event quantity; hardware implementation; FPGA



Academic Editors: Christophe Bobda, Abdessattar Abdelkefi, Riadh Elleuch and Daniil Yurchenko

Received: 17 December 2025

Revised: 15 February 2026

Accepted: 18 February 2026

Published: 20 February 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Inspired by biological visual information systems, event-based vision sensors (EVs) detect changes in external light intensity. When the variable amplitude of light flux on a single pixel reaches a preset threshold, the pixel circuit outputs event pulses through the Address-Event Representation (AER) circuit [1]. The continuous output of event pulses forms an event stream. Compared with traditional image sensors such as Charge-Coupled Device (CCD) and Complementary Metal-Oxide-Semiconductor (CMOS) image sensors, the EVS features asynchronous event stream output, low data volume, high dynamic range, and low latency [2]. In scenarios such as recognition, tracking, and motion trajectory monitoring of high-speed moving targets, it can effectively avoid the resource pressure

caused by the transmission and computation of massive image data, while preventing issues like target motion blur and loss of inter-frame motion information [3], thus boasting broad application prospects.

Although EVSs have significant advantages, noise has emerged as a critical bottleneck hindering their commercialization. On the one hand, due to the complexity of pixel structure, the limitations of semiconductor processes, and the high sensitivity to light intensity changes, EVSs are prone to generating a large number of noise events in complex environments such as low light conditions. Such noise not only occupies limited transmission bandwidth and computing resources, but also seriously interferes with subsequent core applications such as target extraction and tracking [4,5]. On the other hand, the event stream output by EVSs consists of three-dimensional data encompassing pixel coordinates, event timestamps, and polarity information, often presented in the form of 3D scatter plots. This is inherently different from the data structure of traditional frame-based image sensors. If the event streams are simply transformed into 2D images for processing, key temporal information will be lost, making the existing frame image denoising methods completely incompatible. Therefore, developing dedicated denoising methods for event streams has become a core prerequisite for the practical application of EVSs. Based on this, designing an efficient hardware-implementable denoising algorithm for EVS event streams is of great significance for suppressing noise interference, supporting the application of EVSs in multiple fields such as high-speed target detection and tracking, and also serves as a key breakthrough to enhance its commercial value. The three-dimensional image of the event stream of the EVSs is shown in Figure 1.

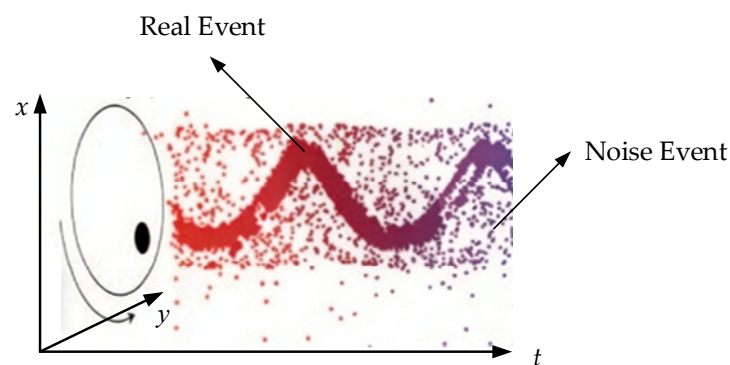


Figure 1. Three-dimensional visualization image of the event stream. The real events and noise events are mixed together. The noise events are widely distributed throughout the image, seriously interfering with applications such as tracking and monitoring of the real events.

After nearly two decades of development, EVS noise suppression research has become a hot research field, and various technical approaches have been formed. These methods exhibit significant differences in principles, performance, and applicability:

The first type of method is “event-frame transformation” denoising, which generates binary images by accumulating event streams within a fixed time window and then reuses traditional image denoising tools for processing. For example, Xie et al. [6] proposed an event frame-based denoising algorithm. This method converts events into binary images by accumulating the event streams over a fixed period of time, and then uses the K-SVD algorithm to denoise the binary images. This method can leverage existing 2D image denoising tools for noise reduction, which is quite convenient and efficient. However, it also has considerable drawbacks. By converting 3D event streams into 2D binary images, this method loses the temporal information inherent in the event streams.

The second type of method is event stream denoising at the software level. For example, Feng et al. [7] proposed a denoising algorithm based on event density. By cal-

culating the event density twice, this algorithm identifies events with low density as noise and filters them out. Zhou et al. [8] proposed a Neighborhood Density-based Spatiotemporal Event Filter (NDSEF), which accumulated events in the time dimension in the form of frames and calculated the neighborhood density. At the same time, the cascade filtering based on NDSEF was realized by adding the pixel window accumulation to achieve the purpose of refined denoising. Miao et al. [9] proposed an event stream denoising algorithm based on dual-threshold multi-layer filtering. The algorithm refined the types of noise, and configured three layers of filter to remove noise step by step according to the characteristics of noise distribution. Specifically, a hot pixel noise filter is applied first to eliminate hot pixel noise, so as to avoid its impact on subsequent filtering processes; a double dimensional filter can be used to remove background activity noise; and a neighborhood density filter is adopted to eliminate scattered random noise. This algorithm makes event stream denoising more refined and accurate, but it also increases the complexity of algorithm implementation.

The third type of method starts from the source of noise and suppresses noise generation by optimizing the pixel circuit structure. The Samsung team proposed several improved Dynamic Vision Sensor (DVS) pixel circuits [1,2]. Through the improvement of pixel circuits, the generation of noise in specific environments can be suppressed. The circuit structure improved by Son et al. [10] can suppress leakage current and thereby reduce the generation of background active noise. This approach can fundamentally solve the noise problem; however, due to the noise-sensitive nature of event cameras, it is rather difficult to improve the circuit, and the denoising effect is comparatively unsatisfactory.

The fourth type of method is hardware-based denoising of event streams. Its core advantage lies in retaining the temporal attribute of the event stream to avoid information loss. Its design is based on the essential feature that “noise events lack spatiotemporal correlation with surrounding events”. The Background Activity Filter (BAF) algorithm is the earliest proposed event stream denoising method [11]. It distinguishes noise from real events by calculating the minimum time difference between the newly arrived event and the events generated in its spatial neighborhood and determining whether this minimum time difference is below the predefined threshold. This method features a straightforward concept and easy implementation; however, its criterion for judging the correlation between events is overly simplistic, resulting in suboptimal denoising performance. Subsequently, some researchers optimized the algorithm and proposed improved denoising methods, such as the Spatiotemporal Correlation Filter (STCF) [12] and the $O(N)$ -Space Spatiotemporal Filter (ONSF) [13]. Guo et al. proposed several new filters based on previous research, including the Fixed Windows Filter (FWF) and the Double Windows Filter (DWF) [14]. The Fixed Windows Filter (FWF) defines a fixed window that refers to the nearest L events that occurred before the current event. It filters out noise by calculating the spatial distance between these historical events and the current event, and updates the fixed window continuously. To avoid the situation where there are too many noise events in the fixed window, making it impossible to determine new events, they proposed the Double Windows Filter (DWF) based on FWF. This algorithm splits the fixed window into a real event window for storing real events and a noise event window for storing noise events. When the minimum spatial distance between a new event and an event in the window is less than the preset threshold, the event is determined to be a real event and the real event window is updated; otherwise, it is determined to be a noise event, with the noise event window being updated. The Multi-Layer Perceptron Filter (MLPF) is a deep neural network filter based on simple MLP [15]. Compared with FWF and DWF, this lightweight classifier trained based on labeled data can achieve better denoising accuracy.

The HashHeat algorithm is a background activity filtering algorithm with $O(L)$ complexity [16]. This algorithm uses a hash function to encode spatiotemporal information, and thus does not store timestamp and location information. It is the first spatiotemporal filter that neither scales with the DVS pixel size N nor stores 32-bit timestamps. The HAST algorithm is a modification of the HashHeat algorithm. It uses a compact two-dimensional binary array to store hash values to maintain the retrieval efficiency of spatiotemporal events within a predefined spatiotemporal volume based on hash functions [17]. This avoids the inefficient event storage of the HashHeat algorithm and the additional arithmetic processing required for detecting temporal correlations, while improving the denoising effect; however, it also increases the consumption of storage space. Zhao et al. [18,19] proposed a Partitioned cache-like Spatiotemporal Filter (PCLF) based on a partitioned cache, achieving synchronous denoising and compression. The PCLF adopts a hierarchical storage structure, using symmetrical multiple sets of cache row and column memory to store event data from partitioned pixel arrays. Zhao et al. [19] implemented this design on an FPGA, with a memory complexity of $O(2N)$, and simultaneously demonstrated capabilities of real-time operation (≤ 60 ns) and low power consumption (< 200 mW). However, its denoising performance is not prominent.

It can be concluded from existing studies that native event stream denoising methods are more compatible with the technical characteristics of EVSs, and that hardware implementation is a key prerequisite for supporting their high-speed dynamic applications.

However, the current hardware-feasible event stream denoising algorithms generally have core contradictions: either they deliver satisfactory denoising performance but incur excessive hardware resource overhead, or they feature controllable resource consumption yet suffer from insufficient denoising accuracy. The optimal balance of “denoise effect–hardware consumption” has not yet been formed. Table 1 summarizes the core performance metrics of several hardware-implemented denoising algorithms, clearly illustrating the differences in storage space and processing latency among existing methods. The storage space consumption is represented by the algorithm’s space complexity, and the resolution of EVSs is set at $N \times N$.

Table 1. Performance of existing hardware-implemented denoise algorithms.

Algorithm	Storage Space	Latency (ns/Event)
BAF [11]	N^2	28
STCF [12]	N^2	45
ONSF [13]	$4N$	33
DWF [14]	L	74
MLPF [15]	$>N^2$	1250
HashHeat [16]	L	120
Hast [17]	$>L$	54
PCLF [18]	$2N$	≤ 60

Based on the core problem that a large amount of noise exists in the event stream generated by EVSs, which prevents the effective utilization of event stream data, this paper proposes an EVS event stream denoising algorithm based on local spatiotemporal event quantities and realizes hardware deployment on the FPGA platform. The core objectives of this research are as follows: on the premise of ensuring algorithm real-time performance and controllable hardware resource consumption, significantly improve event stream denoising accuracy to break through the constraints of noise interference on EVS applications, and provide key technical support for their practical application in visual navigation tasks such as high-speed target detection and tracking. This paper innovatively proposes a two-stage denoising algorithm based on local spatiotemporal event quantities and completes its

hardware implementation. While achieving excellent denoising performance, the algorithm does not lead to a significant increase in resource overhead, thus effectively accomplishing the core research objectives.

2. Event Stream Noise Analysis and Denoising Algorithm Design

2.1. Event Stream Noise Analysis

The common types of noise in traditional image sensors include Gaussian noise, shot noise, reset noise, hot pixel noise, etc. [20–24]. These types of noise are often mixed, forming incorrect gray values that are superimposed on the real gray values, causing some pixels to fail to accurately represent light intensity information. For EVSs, the semiconductor devices and circuit structure design they adopt, as well as external interference, will inevitably generate noise [25–28]. Noises such as shot noise, reset noise, and hot pixel noise also exist in EVSs. The most significant difference is that EVSs output event streams, and each event can be represented by $e(x, y, p, t)$. It can be seen that each event contains four elements: (x, y) are the coordinates of the pixel that generates the event, p is the polarity of the event, and t is the timestamp of the event. And when each event accumulates, it forms an event stream. Intuitively, it is a discrete point in three-dimensional spacetime, where the three dimensions are the horizontal coordinate, vertical coordinate and the timestamp respectively. Each point in three-dimensional spacetime represents an event. The difference between the outputs of EVSs and traditional image sensors is shown in Figure 2.

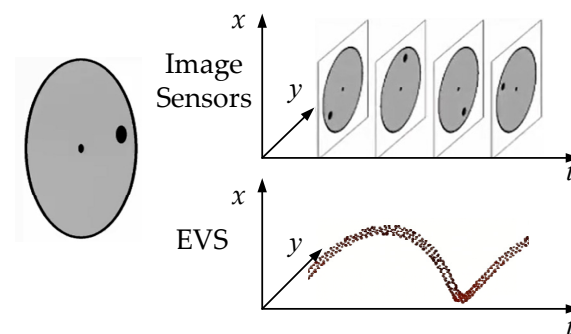


Figure 2. Three-dimensional schematic diagram of event stream.

To better study the spatiotemporal distribution characteristics of noise events, Khodamoradi et al. [13] conducted noise modeling. Although the noise sources of different EVS may vary, the basic noise forms are the same, such as shot noise, amplifier noise, reset noise, etc. Therefore, the modeling of noise events can still represent the spatiotemporal characteristics of most noises. They used a DVS128 camera to capture event streams in a fixed scene and conducted statistical modeling on the noise within the event streams. The statistical results showed that background activity noise can be described by the Poisson distribution [13]:

$$P\{N(t) = n\} = \frac{(\lambda t)^n}{n!} e^{-\lambda t} \quad (1)$$

where λ represents the mean value of noise events generated by each pixel on average, $N(t)$ represents the number of noise events generated by a single pixel within the time period t , and P represents the probability that a single pixel generates n noise events within the time period t .

It can be seen from Equation (1) that within a fixed time period t , the more background activity noise a pixel unit generates, the lower the probability of its occurrence. This indicates that the probability of noise events occurring densely in the same pixel unit is very small, and they usually occur at random times and random positions with an uncertain

probability. Therefore, the spatiotemporal distances between noise events are generally large, and their distribution in the event stream is relatively sparse. The occurrence of real events is due to the movement of objects or changes in light intensity. In such cases, the activated pixel units are generally adjacent, so the events generated by adjacent pixel units should also be adjacent in terms of spatiotemporal distribution. Therefore, within the spatiotemporal domain of the event stream, noise events are sparsely distributed with large spatiotemporal distances, resulting in a small number of other events in their spatiotemporal neighborhoods. However, real events generated by real targets are densely distributed, resulting in a larger number of other events in the spatiotemporal neighborhood of real events.

In conclusion, due to the uncontrollability of the factors generating noise events, the generation of noise events is also random, resulting in a scattered and sparse spatiotemporal distribution of noise events. The generation of real events is due to the movement of objects or changes in light intensity, and the events produced by adjacent pixel units should also be adjacent in spatiotemporal distribution, with a relatively dense distribution. Therefore, in the entire event stream, all events fall into only two categories: real events and noise events. Real events have spatiotemporal correlation, while noise events are generally random and isolated. Therefore, the event stream denoising algorithm precisely identifies noise through the spatiotemporal distribution characteristics of events and noise, ultimately eliminating the influence of noise on real events.

2.2. Design of Denoising Algorithm

Through the analysis of the generation mechanism and distribution characteristics of noise events and real events, it can be found that for real events, there is a spatiotemporal correlation among real events. As for noise events, they are generally random and isolated. Based on these spatiotemporal characteristics, a denoising method based on local spatiotemporal event quantities is proposed.

2.2.1. Local Spatiotemporal Event Quantities

The so-called local spatiotemporal domain refers to defining a spatiotemporal neighborhood centered on a new event when it arrives. The specific implementation method is as follows: first, a spatial neighborhood is selected with the position of the pixel unit where the event occurs as the center, and this spatial neighborhood contains $L \times L$ pixel units. Meanwhile, the timestamp of the event is taken as the end point of the temporal neighborhood, and a time period Δt is traced back forward. The representation of the local spatiotemporal domain is shown in Figure 3.

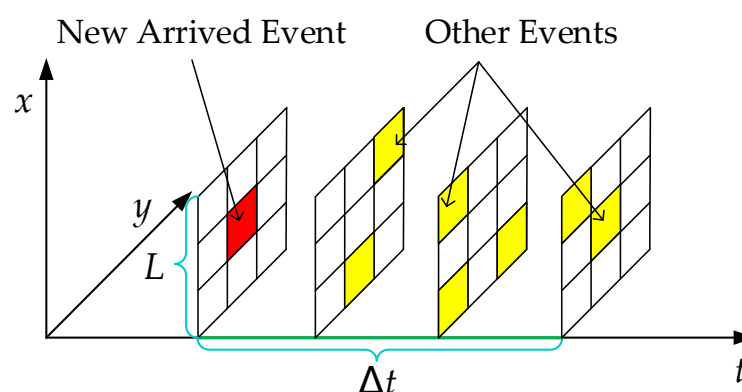


Figure 3. Schematic diagram of local spatiotemporal domain.

The number of events in $L \times L \times (\Delta t + 1)$ pixel cells in the spatiotemporal neighborhood selected by the newly arrived event is calculated, and the result is the local spatiotemporal event quantity d :

$$d = \sum_i \sum_j \sum_{t=t_0-\Delta t}^{t_0} \gamma(x_0 - \frac{L+1}{2} + i, y_0 - \frac{L+1}{2} + i) \cdot \gamma(t) \cdots \cdots 1 \leq i, j \leq L \quad (2)$$

where x_0, y_0 are the spatial coordinates of the new arrived event, $\gamma(x, y)$ is a binary function, indicating whether there is an event in a pixel cell. If so, $\gamma(x, y)$ is 1; otherwise, $\gamma(x, y)$ is 0. Therefore, $\gamma(x, y)$ denotes whether an event occurs at a pixel cell at time t , and d is the total number of events that occur in the whole spatiotemporal neighborhood of the newly arrived event.

The local spatiotemporal event quantity d represents the number of events in the neighborhood of a new event, so that the degree of event correlation can be judged. There is spatiotemporal correlation between real events, and their spatiotemporal distribution is also dense, while noise events occur randomly, so their spatiotemporal distribution is scattered and sparse. Therefore, if an event is a real one, there are relatively many events in its spatiotemporal neighborhood, so its local spatiotemporal event count is relatively high. Conversely, if it is a noise event, there are relatively few events in its spatiotemporal neighborhood, so its local spatiotemporal event count is also relatively low. Therefore, the local spatiotemporal event quantity can represent the correlation degree between events. By comparing with the set threshold, real events and noise events can be distinguished.

2.2.2. Design of Denoising Algorithm Based on Local Spatiotemporal Event Quantity

Based on the generation mechanism and spatiotemporal characteristics of noise, noise can be mainly divided into two categories, background activity noise and hot pixel noise [29]. The noise in the event stream is mainly background activity noise, which occurs randomly across different pixel units and at different times. It is widely distributed in the spacetime of the entire event stream with a relatively sparse distribution. In contrast, hot pixel noise is mainly caused by defective pixels, i.e., some pixel units periodically generate events even when there is no change in light intensity. Therefore, hot pixel noise only appears periodically at fixed pixel positions.

By analyzing the generating mechanism and spatiotemporal characteristics of noise, a denoising method based on local spatiotemporal event quantity is proposed. First, it addresses the background activity noise which is sparsely distributed in the spatiotemporal domain. From the previous analysis, it can be concluded that real events are surrounded by a large number of other events in their spatiotemporal neighborhoods, and noise events have far fewer adjacent events in their spatiotemporal neighborhoods. In other words, real events have a high local spatiotemporal event count, while noise events have a low one. Therefore, after selecting the event stream to be denoised, the spatiotemporal neighborhood parameters of the local spatiotemporal event quantity and the noise judgment threshold ψ are determined. For each input event, the value of its local spatiotemporal event quantity d is calculated and compared with the noise threshold. If $d \geq \psi$, the event is judged to be a real event; otherwise, it is classified as a noise event.

Then, consideration is given to the removal of hot pixel noise, which can be approached by analyzing the spatial relationship between hot pixel noise events and adjacent pixel positions. After filtering the background activity noise, it can be considered that there are only real events and hot pixel noise in the event stream. Hot pixel noise appears at a fixed position and is sparsely distributed in space, while real events are relatively densely distributed in space. Therefore, hot pixel noise can be filtered out by leveraging the spatial relationship between an event and its adjacent pixel positions. The specific method is as

follows: in the fixed time neighborhood t , check whether events occur at the pixel positions adjacent to the new event to obtain a judgment value R . If such events exist, the new event is determined to be a real event; otherwise, it is classified as hot pixel noise.

In this paper, the parameter values used in the denoise algorithm are $L = 5$, $\Delta t = 5$ ms, and the noise threshold is 5. The specific experimental flowchart is shown in Figure 4.

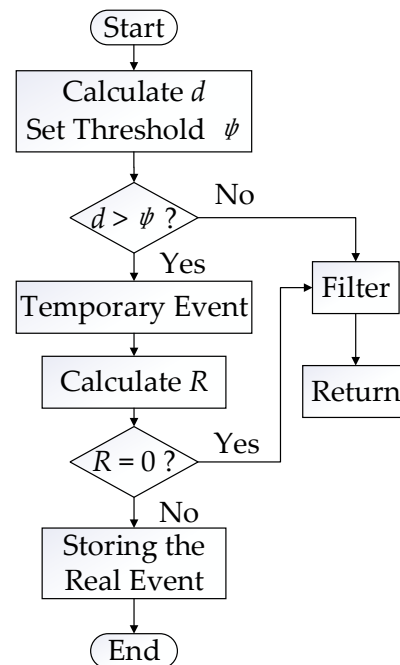


Figure 4. Design flowchart of the denoising algorithm.

2.3. Design of Hardware Implementation of Denoising Algorithm

Compared with exporting event stream information, denoising in software and then importing it into a back-end system for application analysis of event stream information, the hardware implementation of the denoising algorithm can realize the analysis and processing of event stream information more conveniently and quickly, and better exert the temporal attributes of event stream information, so as to deal with scenes that need real-time event information, such as autonomous driving, high-speed moving target trajectory monitoring and other application fields that require real-time analysis of information [30–33].

For the hardware porting of the event stream denoising algorithm based on local spatiotemporal event quantity, the Xilinx ZYNQ7000 (Santa Clara, CA, USA) development board is selected. The core board is mainly composed of the minimum system of the XC7Z020 chip, two DDR3 memory modules and a QSPI FLASH, which undertake the functions of high-speed data processing and storage for the ZYNQ system. Comprehensive design is carried out by the ZYNQ7000 platform and Vivado software (version 2017.4), and performance indicators such as power consumption, hardware resource consumption and latency are measured by simulation. In order to verify the design concept and serve as a reference model for hardware implementation, simulations are first carried out in Matlab (version R2017b), and the results show that the designed algorithm can achieve effective denoising. Hardware simulation design was then carried out, and the hardware performance was completely consistent with the Matlab simulator.

In the hardware implementation, some representative event contents are selected for testing. The test data consists of event streams with sparse noise and event streams with dense noise. Firstly, in the data acquisition, event stream data was collected by the EVK4 camera developed by Prophesee (Paris, France). This event camera features

a resolution of 1280×720 , a dynamic range of >86 dB, and a maximum output rate of 1.6 Gbps. The event camera can decode the complete event stream information through the supporting Metavision software (version 4.0). Appropriate event stream segments were selected from the decoded information, converted into coe files, and transmitted to the FPGA for storage. Then, the circuit design of the denoising algorithm was completed using the hardware description language. After the comprehensive design was finished, simulation and verification were carried out to confirm the design scheme. Finally, the circuit design results were downloaded to the FPGA, and event stream denoising was performed on the FPGA. After denoising, the denoised event stream was output, and the storage space consumption and latency of the algorithm were measured.

The working clock frequency of the denoising algorithm is set to 100 MHz. As shown in Figure 5, this is an FPGA hardware architecture designed for the two-step denoise algorithm based on local spatiotemporal event quantities, aiming to implement noise reduction processing for event streams. Within the sampling control unit, the time window Δt is controlled by a timer, the pixel coordinates of events are located via row and column selection, and the neighborhood selection unit defines the $L \times L$ spatial neighborhood range. Eventually, all event data within the $L \times L \times \Delta t$ spatiotemporal neighborhood are extracted to provide input for subsequent denoise calculations. In the denoising unit module, the neighborhood event data buffer module temporarily stores the neighborhood data. For the extracted neighborhood event data, the total number of events within the local spatiotemporal range is counted (event count calculation). Then, the event determination unit compares the statistical value with a preset threshold to classify the event as either a “real event” (retained) or a “noise event” (filtered out). Upon completion of the first-step denoise, the denoise unit module is invoked again to determine hot pixel noise: the data temporarily stored in the buffer module is cleared. The system then waits for the registration of the neighborhood event stream of new events, and repeats the steps of the denoise unit module to implement the second-step denoise. Finally, the denoised event data is output through the interface circuit. The data buffer is used for temporarily storing the denoised valid event stream, the interface circuit implements data interaction between the hardware and external devices (camera and host computer), and the system control module provides global timing, reset, and parameter configuration signals.

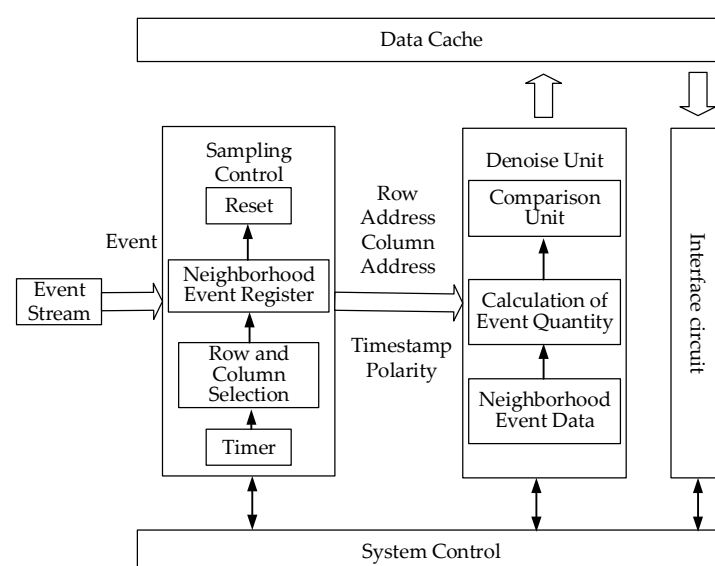


Figure 5. Hardware implementation architecture of denoising algorithm.

The function modules of the denoising unit in the hardware architecture are mainly realized by the state machine, which mainly includes calculating the local event quantity of the newly arrived events, judging the nature of the event by comparing it with the noise threshold, and outputting a flag bit to determine whether the event is a noise event. The specific state transition process of the state machine is shown in Figure 6.

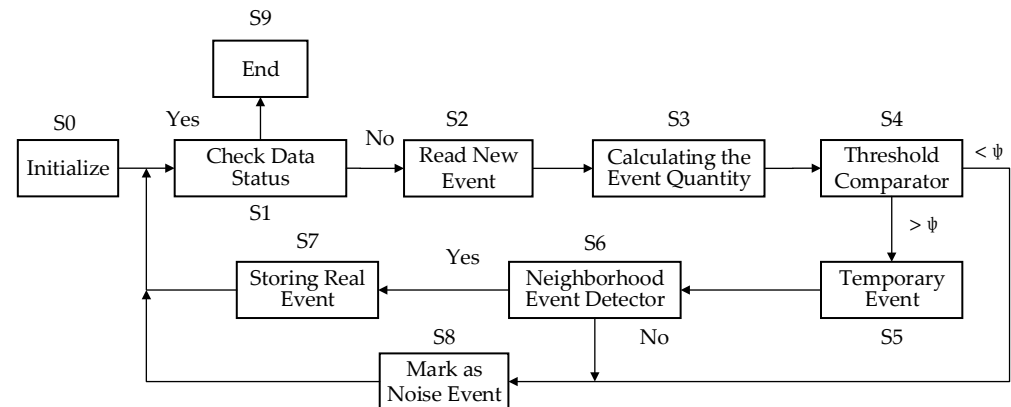


Figure 6. State transition process of the state machine.

As shown in Figure 6, all modules such as FIFOs, counters, clock signals and RAM are initialized in state S0, and the state machine jumps to S1. In state S1, if the next signal to be read is detected as an empty signal, i.e., the empty logic level is 1, it means that there is no data input at this time, and the data transmission is over, so the state machine jumps to S9; otherwise, the state jumps to S2. In state S2, the read enable signal of the RAM is activated, the cached data is read out and assigned to the register, and the state jumps to S3. In state S3, the read enable signal of the RAM is deactivated, the neighborhood event register is activated, the number of events within the spatiotemporal neighborhood of the newly read event in the neighborhood event register is calculated, the counter records the calculation result, and the state jumps to S4. In state S4, the counter result is compared with the preset noise threshold. If it is less than the threshold, the event is determined to be noise, and the state machine jumps to S8; otherwise, it is determined to be a real event, the event data is cached, and the state jumps to S5. The state S5 is to cache the event data and re-determine its neighborhood data. In state S6, it is determined whether there is data in the new neighborhood event register of the event in the register that meets the neighborhood requirements. If so, it is judged to be real data, and the state jumps to S7. Otherwise, it jumps to S8. In state S7, the write enable signal of the denoised event RAM is activated, the register data is written in, and the state jumps to S1. In state S8, the write enable signal of the noise event RAM is activated, the data is written into it, and the state jumps to S1. In state S9, the clock input is stopped and the data is cleared. The filtering of noise events is completed through the transition of the state machine, and the denoised real event stream is output.

3. Analysis of Hardware Implementation Results of Denoising Algorithm

3.1. Visualized Images of the Hardware Implementation Results of Denoising Algorithm

To further study the denoising capability of the denoising algorithm proposed in this paper, the data of event streams of two scenarios are selected as test data, namely the event streams with sparse noise and dense noise. Three algorithms were chosen as control groups to explore the denoising effect of the proposed algorithm on event streams in different noise environments. The comparison algorithms selected here are STCF, ONSF and DWF, respectively.

Scene 1 is shown in Figure 7a, which is a scene of pedestrians walking on a rainy road. This scene was captured from a distance. Due to favorable lighting conditions during the daytime, there are relatively fewer noise events in the event stream. This event stream is used as low-noise event stream data to test the denoising capability of the denoising algorithm in low-noise event streams.

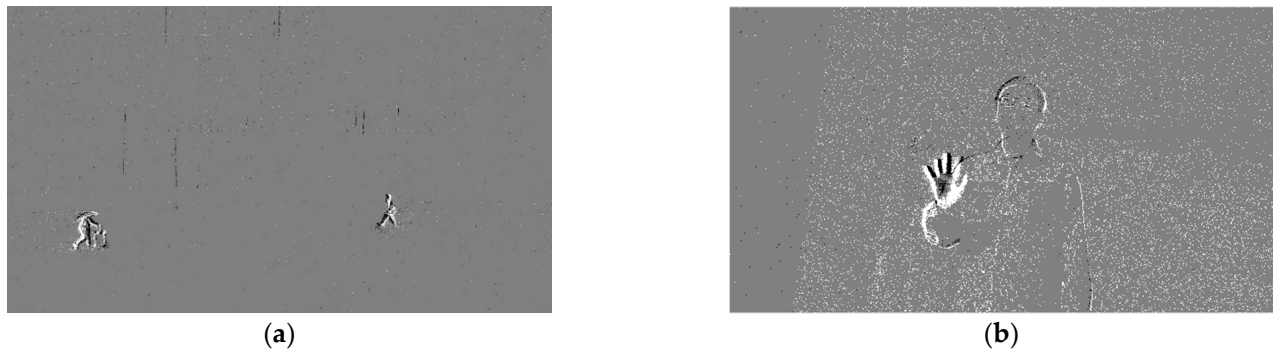


Figure 7. Two-dimensional images of event streams in the two scenes. (a) The original image of the event stream of pedestrians on a rainy road; (b) The original image of the event stream of the character waving his hand.

As shown in Figures 8–11, through comparison, it can be seen that there is basically no noise in the images denoised by the algorithm proposed in this paper, the STCF and the ONSF, and the noise events are basically filtered out. However, some noise still remains in the images denoised by the DWF algorithm. Meanwhile, in the images with event streams filtered out, the algorithm in this paper and the DWF algorithm filter out relatively few real events, and only a small number of events representing human contours are filtered out. In contrast, STCF and ONSF filter out a large number of real events, and the contours of people are relatively obvious in the images of the event streams filtered by them. To sum up, when dealing with low-noise event streams, the denoising effect of the algorithm proposed in this paper is relatively good, which filters out noise events while retaining real events as much as possible.

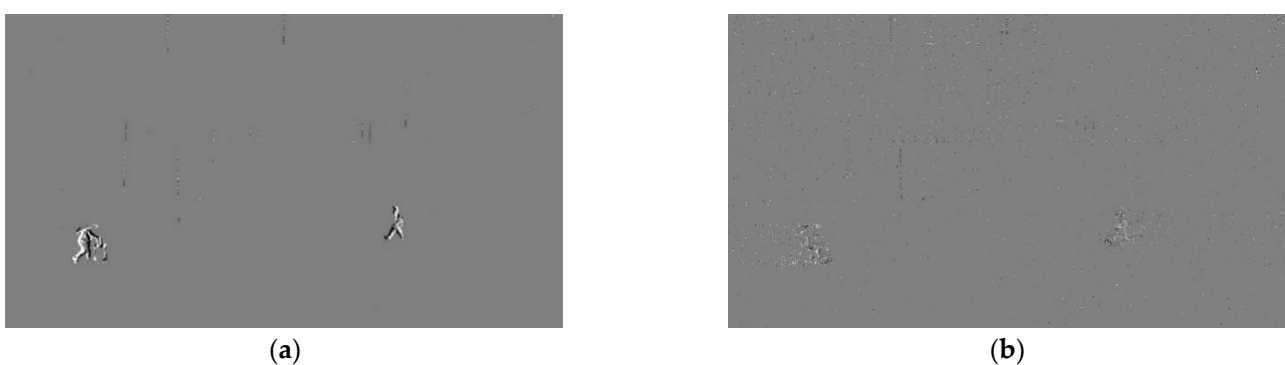


Figure 8. Two-dimensional visualization images of the denoising results using the denoising algorithm proposed in this paper for the event stream of the scene where pedestrians walk on a rainy road. (a) 2D image of the denoised event stream, with only a very small number of noise events presenting in the image, and the outlines of the characters remaining clear; (b) 2D image of the filtered event stream; a large number of noise events have been filtered out, but the real events representing the contours of the characters have been filtered out very little.

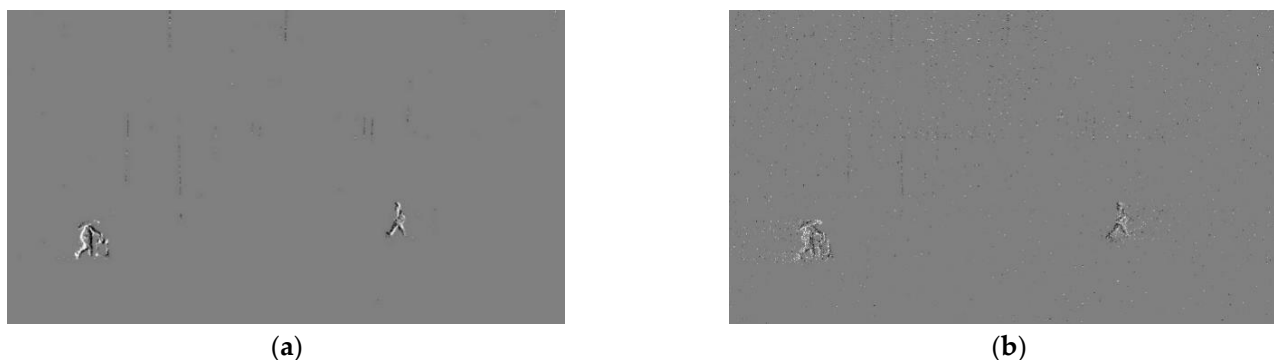


Figure 9. Two-dimensional visualization images of the denoising results using STCF for the event stream of the scene where pedestrians walk on a rainy road. (a) 2D image of the denoised event stream, with a significant amount of noise events being filtered out; (b) 2D image of the filtered event stream, representing that a large number of the real events of the character outline have been filtered out.

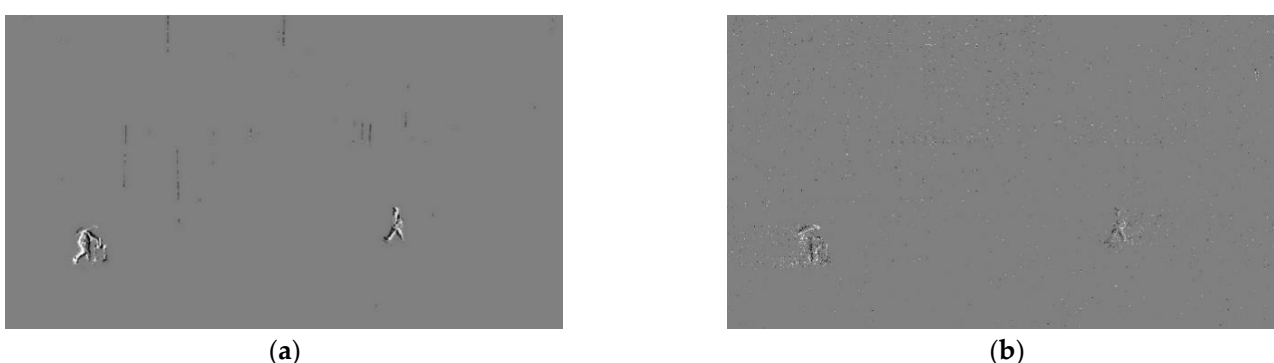


Figure 10. Two-dimensional visualization images of the denoising results using ONSF for the event stream of the scene where pedestrians walk on a rainy road. (a) 2D image of the denoised event stream, with only a very small number of noise events remaining; (b) 2D image of the filtered event stream; the small parts of the real events that represent the outlines of the characters have been filtered out.

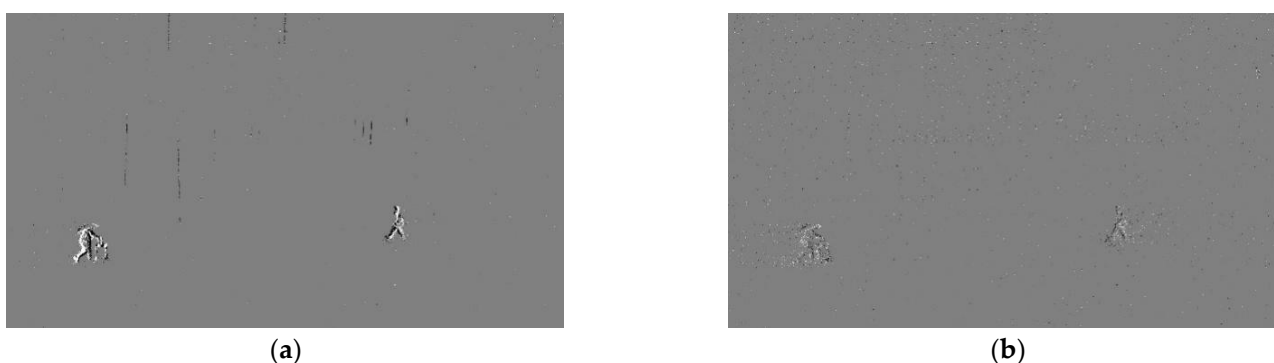


Figure 11. Two-dimensional visualization images of the denoising results using DWF for the event stream of the scene where pedestrians walk on a rainy road. (a) 2D image of the denoised event stream, with a few noise events remaining and the human figure outline preserved completely; (b) 2D image of the filtered event stream, with only a few real events representing the human figure outline being filtered out.

Scene 2 is shown in Figure 7b, which is a scene where the target person waves his hand at night. Since it was shot in low light conditions at night, there are a large number of noise events in the event stream. It can be seen from the figure that the noise events are widely distributed throughout the entire image. This event stream is used as high-noise

event stream data to test the denoising capability of the denoising algorithm in high-noise event streams.

As shown in Figures 12–15, it can be seen from the 2D images of the denoised event streams that there are still some noise events that have not been filtered out in the images. Among them, the images denoised by the algorithm in this paper have the fewest noise events, while those denoised by the STCF and ONSF algorithms have relatively more noise events. It can be seen from the 2D images of the filtered event stream that the algorithm in this paper filters out the fewest real events, and the character contours in the images of the filtered event stream by the ONSF algorithm are the most obvious. In summary, in high-noise event streams, the denoising capability of the algorithm proposed in this paper is also relatively excellent.

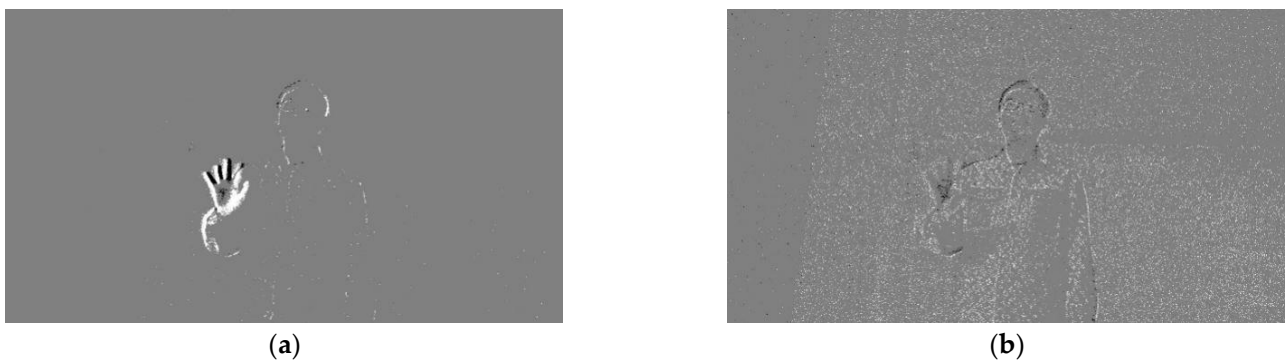


Figure 12. Two-dimensional visualization images of the denoising results using the denoising algorithm in this paper for the event stream of the scene where the target character waves at night. (a) 2D image of the denoised event stream, in which there are only a few noise events and the human figure outline is clearly retained; (b) 2D image of the filtered event stream; a large number of noise events have been removed, but the real events representing the contours of the characters have been filtered out very little.

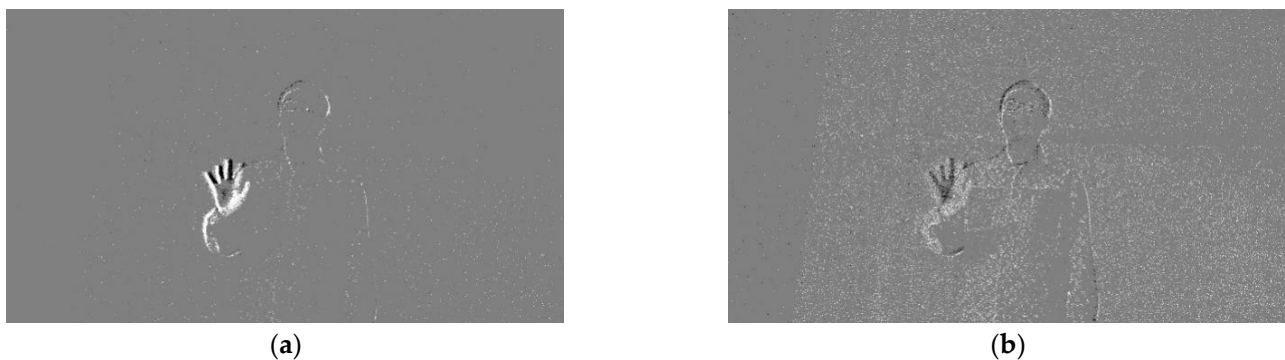


Figure 13. Two-dimensional visualization images of the denoising results using STCF for the event stream of the scene where the target character waves at night. (a) 2D image of the denoised event stream, with still many noise events present in the image; (b) 2D image of the filtered event stream, with a large number of noise events being filtered out.

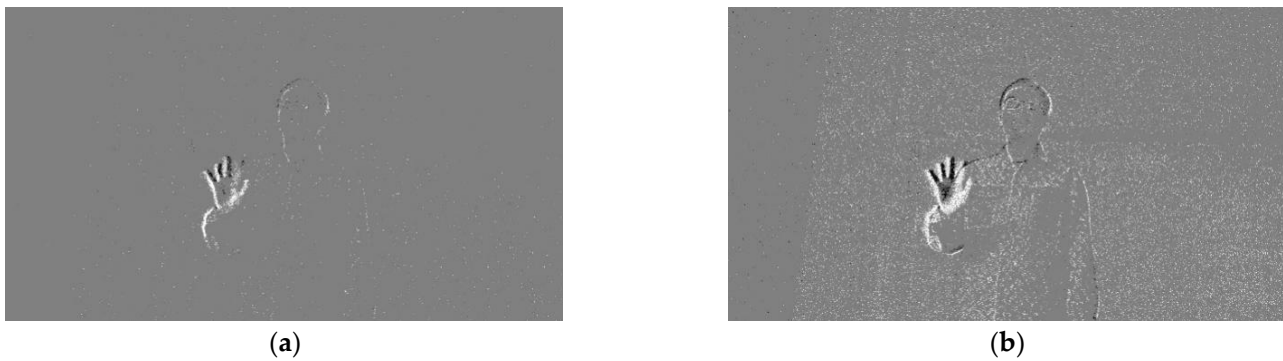


Figure 14. Two-dimensional visualization images of the denoising results using ONSF for the event stream of the scene where the target character waves at night. (a) 2D image of the denoised event stream, in which there are still many noise events and the human figure outline is not clear; (b) 2D image of the filtered event stream, a large number of real events representing the human figure outline have been filtered out.

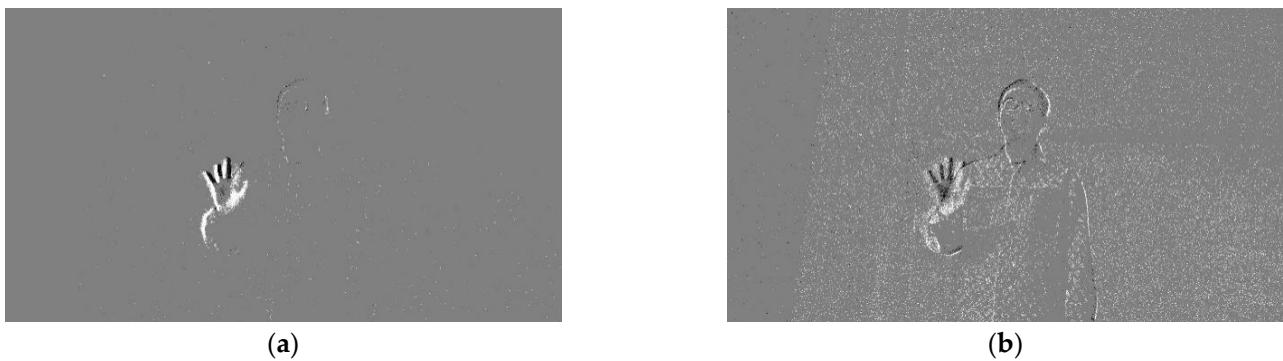


Figure 15. Two-dimensional visualization images of the denoising results using DWF for the event stream of the scene where the target character waves at night. (a) 2D image of the denoised event stream, where the outlines of the characters are not clear; (b) 2D image of the filtered event stream. Most of the noise events have been removed, but a large number of real events representing the contours of the characters have also been filtered out.

3.2. Evaluation of Denoising Effectiveness and Resource Consumption

3.2.1. Evaluation of Denoising Effectiveness

To more intuitively observe the denoising effect, this paper selects an evaluation method that can quantify the denoising effect. After comprehensively comparing several denoising evaluation methods, the evaluation method based on event probability is chosen here for the evaluation of the denoising effect [34,35]. The quantification of the spatiotemporal correlation degree between an event and other events can be used to represent the probability that the event is a real event generated by the target [29]. The calculation formula for this probability can be expressed as:

$$P(e_0) = \sum_i \frac{1}{2\pi\sigma_1\sigma_2} \cdot e^{-\left(\frac{d_i^2}{2\sigma_1^2} + \frac{\tau_i^2}{2\sigma_2^2}\right)} \quad (3)$$

$$d_i = \sqrt{(x_i - x_0)^2 + (y_i - y_0)^2} \quad (4)$$

$$\tau_i = t_i - t_0 \quad (5)$$

where $P(e_0)$ represents the real event probability of the event. When this probability exceeds a certain threshold, the event is recognized as a real event. Parameters σ_1 and σ_2

are the variances of the distance d between events and the time difference τ between events, respectively. Through the analysis of this function and the spatiotemporal neighborhood parameters, it is found that excessively large or small values of the two parameters will affect the discrimination of noise. Therefore, $\sigma_1 = 1$ and $\sigma_2 = 2$ are adopted for the discrimination of noise events.

The noise probability threshold is the average of the real event probabilities of all events in the region of the event stream that is entirely composed of noise events. The calculation formula can be expressed as:

$$P_{ave} = \frac{\sum_i \frac{1}{2\pi\sigma_1\sigma_2} \cdot e^{-\left(\frac{d_i^2}{2\sigma_1^2} + \frac{\tau_i^2}{2\sigma_2^2}\right)}}{N} \quad (6)$$

where P_{ave} is the threshold for determining noise, and N is the total number of statistical noise events. When the degree of event correlation in the event stream is higher than P_{ave} , the event is a real event. Conversely, if it is lower than or equal to P_{ave} , the event is regarded as a noise event.

After selecting the evaluation method, two evaluation indicators for the denoising effect are proposed here:

- (1) The Noise Event Ratio (NER) is defined as the ratio of noise events in the event stream after denoising. The NER index characterizes the denoising capability of the denoising algorithm. The fewer noise events in the event stream after denoising, the smaller the NER value.
- (2) The Event Noise Ratio (ENR) refers to the ratio of real events retained in the filtered noise event stream. The ENR index characterizes the capability of the denoising algorithm to retain real events. The fewer real events in the filtered event stream, the smaller the ENR value.

The two indicators are mutually exclusive. When the noise determination threshold is set too high, causing the event stream to be overly denoised, the vast majority of noise events will be filtered out, resulting in a very low NER; however, some real events will also be wrongly filtered out, leading to a high ENR. On the contrary, when the noise determination threshold is set too small, in event streams where denoising is not fully successful, NER will be high while ENR will be low. Therefore, when evaluating the denoising effect, the results of both indicators should be analyzed comprehensively. The method with both indicators being very low has the best effect.

Then, the denoising results of the denoising algorithms based on local spatiotemporal event quantities in this paper, STCF, ONSF and DWF, will be quantified respectively, and the effects of the denoising algorithms will be demonstrated through the NER and ENR indicator values.

As can be seen from Figure 16, the NER value of the denoising algorithm in this paper is 8.37%, and the ENR value is 25.10%. The NER of the STCF algorithm is 24.50%, and its ENR value reaches 24.92%. The NER of the ONSF algorithm is 29%, and its ENR is also relatively high, reaching 36.27%. The NER of the DWF algorithm is 11.58%, and its ENR is also relatively high, reaching 32.55%.

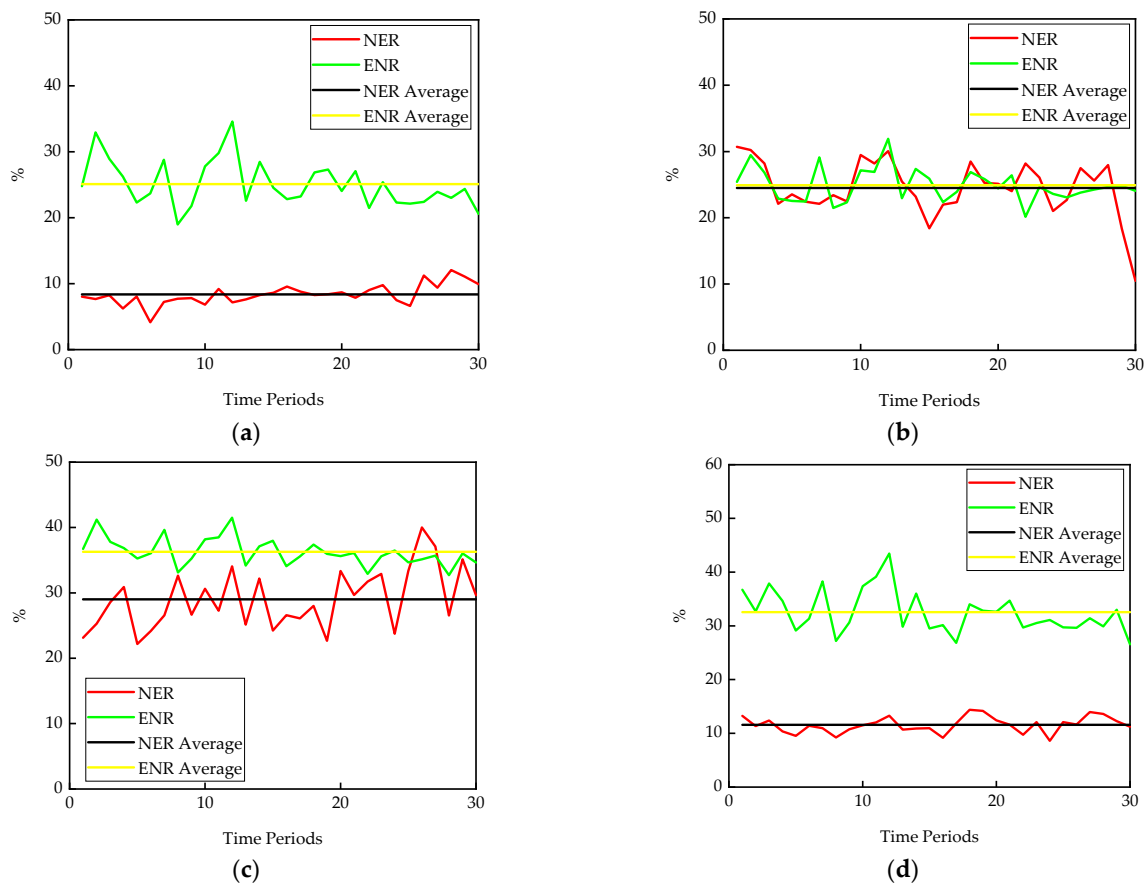


Figure 16. Results of evaluation indicators for denoising algorithms. (a) Indicator values for the proposed algorithm in this paper; (b) Indicator values for the STCF; (c) Indicator values for the ONSF; (d) Indicator values for the DWF.

From the data analysis results, the two indicators of the denoising algorithm in this paper are relatively low. This indicates that the algorithm basically filters out the noise events, and at the same time, a small number of real events representing the target edge and contour are filtered out. In the comparison algorithms, the NER index results of the STCF and ONSF algorithms are relatively large, indicating that there is still a large amount of noise in the event stream after denoising. The ENR results of the ONSF and DWF algorithms are also relatively large, indicating that the two algorithms filter out a large number of real events during the denoising process. In conclusion, through the above comparative analysis, it can be found that the denoising algorithm in this paper has the best denoising effect.

3.2.2. Resource Consumption of Hardware Implementation

After the hardware implementation of the denoising algorithm is completed and the denoising of the event stream is implemented, the hardware resource consumption of the denoising algorithm can be measured. Here, the two main indicators considered are latency and storage space.

In the measured resource overhead, storage space is represented by the number of 32-bit storage units. The resolution is represented by $N \times N$, and the usage of storage space is related to the resolution of the event stream. By comparing the hardware resource overhead, it can be found that the latency of the denoising algorithm in this paper is relatively large, reaching 110 ns. This is because the algorithm design adopts a two-step denoising method. After filtering out the background active noise by using the local spatiotemporal event counts, it further removes hot pixel noise based on its isolated characteristics. There-

fore, more time will be spent on denoising. The other several algorithms, on the other hand, filter out events in one step based on their temporal or spatial relationships, resulting in lower latency for these algorithms.

In terms of storage space, since the ONSF algorithm adopts a more storage-efficient design method, it requires two storage units in both the row and column directions of the pixel units, totaling $4N$ storage units. Therefore, its storage space is relatively small. The DWF algorithm uses a fixed event window for denoising, and the length of the window is L , so L storage units are required. However, the other two algorithms both require a storage unit to be allocated for each pixel unit, so the algorithm requires relatively more storage units. The results of the evaluation indicators for the denoise algorithms are shown in Table 2.

Table 2. Results of the evaluation indicators of the denoising algorithm and hardware resource consumption.

Algorithm	NER	ENR	Latency (ns/Event)	Storage Space
STCF [12]	24.50%	24.92%	45	N^2
ONSF [13]	29.00%	36.27%	33	$4N$
DWF [14]	11.58%	32.55%	74	L
This Algorithm	8.37%	25.10%	110	N^2

From the results of denoising indicators and resource consumption, it can be seen that the denoising algorithm in this paper achieves excellent denoising effects while consuming the average level of storage space and causing significant latency.

4. Conclusions

This paper proposes a denoising algorithm based on local spatiotemporal event quantities and conducts systematic verification focusing on the algorithm's performance and engineering implementation. From the intuitive comparison results of denoised images, it can be seen that this algorithm can effectively filter out noise events and achieve high-quality denoising effects. After completing the hardware implementation of the denoising algorithm in this paper, the latency and the consumption of storage space of the algorithm were measured. An event probability-based effect evaluation method was adopted to conduct quantitative analysis of the denoising performance through two core metrics: NER and ENR. A comprehensive analysis of latency, storage space consumption, and quantitative evaluation results shows that although the denoising algorithm in this paper has a slight increase in resource consumption, it achieves better noise filtering performance. Its overall performance meets the expected design goals, providing a solution with both effectiveness and engineering feasibility for noise processing in EVSs.

The current methodology still has certain limitations. Although the data in this study were acquired using the EVK4 event camera, real-time data transmission between the EVK4 camera and the hardware platform was not achieved. Moreover, despite the excellent denoising performance of the proposed algorithm, there remains room for optimization in terms of hardware resource overhead. These aspects also point out the directions for future research work. In future studies, further optimizing the algorithm to reduce hardware resource overhead and processing latency will be an important research direction for improving this work. The development of EVS denoising technology is bound to promote broader and more extensive applications of event-based vision sensors. Meanwhile, researching and implementing real-time data transmission between the EVK4 camera and hardware will also be a crucial step. Realizing real-time transmission and denois-

ing between the camera and hardware will provide a solid guarantee for the commercial application of EVSs.

Author Contributions: Conceptualization, D.G. and X.J.; methodology, D.G.; validation, D.G.; formal analysis, D.G.; investigation, D.G.; resources, Y.Y. and X.J.; data curation, D.G.; writing—original draft preparation, D.G.; writing—review and editing, Y.Y. and X.J.; visualization, D.G.; supervision, Y.Y. and X.J.; project administration, Y.Y. All authors have read and agreed to the published version of the manuscript.

Funding: The project supported the Fundamental Research Funds for the Central Universities (104972025KFYjc0086).

Informed Consent Statement: Not applicable.

Data Availability Statement: Dataset available on request from the authors.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Yunjae, S.; Seungnam, C.; Masamichi, I.; Jeongseok, K.; Youngho, L. A 1280×960 Dynamic Vision Sensor with a 4.95μm Pixel Pitch and Motion Artifact Minimization. In Proceedings of the 2020 IEEE International Symposium on Circuits and Systems (ISCAS), Seville, Spain, 12–14 October 2020; Volume 10, pp. 1–5. [CrossRef]
2. Li, C.H.; Longinotti, L.; Corradi, F.; Delbruck, T. A 132 by 104 10μm-Pixel 250μW 1kefps Dynamic Vision Sensor with Pixel-Parallel Noise and Spatial Redundancy Suppression. In Proceedings of the 2019 Symposium on VLSI Circuits, Kyoto, Japan, 9–14 June 2019. [CrossRef]
3. Gallego, G.; Delbrück, T.; Orchard, G.; Bartolozzi, C.; Taba, B.; Censi, A.; Leutenegger, S.; Davison, A.J.; Conradt, J.; Daniilidis, K.; et al. Event-Based Vision: A Survey. *IEEE Trans. Pattern Anal. Mach. Intell.* **2022**, *44*, 154–180. [CrossRef] [PubMed]
4. Kryjak, T. Event-Based Vision on FPGAs—A Survey. In Proceedings of the 27th Euromicro Conference on Digital System Design (DSD), Paris, France, 28–30 August 2024. [CrossRef]
5. Wolrige, S.H.; Howe, D.; Majidiyan, H. Intelligent Computerized Video Analysis for Automated Data Extraction in Wave Structure Interaction; A Wave Basin Case Study. *J. Mar. Sci. Eng.* **2025**, *13*, 617. [CrossRef]
6. Xie, X.; Du, J.; Shi, G. DVS Image Noise Removal Using k-SVD Method. *Int. Conf. Graph. Image. Process.* **2018**, *10*, 106–153. [CrossRef]
7. Feng, Y.; Lv, H.; Liu, H.; Zhang, Y.; Xiao, Y.; Han, C. Event Density Based Denoising Method for Dynamic Vision Sensor. *Appl. Sci.* **2020**, *10*, 2024. [CrossRef]
8. Zhou, X.L.; Bei, C.; Zhang, N.; Xin, X. Event-based Camera Spatial Target Event Stream Denoising Algorithm. *Infrared Laser Eng.* **2023**, *52*, 326–335. [CrossRef]
9. Miao, H.T.; Li, K. Research on Event Stream Denoising Algorithm Based on Double-Threshold Multi-Layer Filtering. *Inform. Technol. Inform.* **2025**, *11*, 54–59. Available online: https://www.sdie.org.cn/list_57/1125.html (accessed on 12 October 2025).
10. Son, B.; Suh, Y.; Kim, S.; Jung, H.; Kim, J.S.; Shin, C.; Park, K. 4.1 A 640×480 Dynamic Vision Sensor with A 9μm Pixel and 300Meps Address-event Representation. In Proceedings of the 2017 IEEE International Solid-State Circuits Conference, San Francisco, CA, USA, 5–9 February 2017. [CrossRef]
11. Delbruck, T. Frame-Free Dynamic Digital Vision. In Proceedings of the International Symposium on Secure-Life Electronics Advanced Electronics for Quality Life and Society 2008, Tokyo, Japan, 6–7 March 2008. [CrossRef]
12. Liu, H.; Brandli, C.; Li, C.; Liu, S.C.; Delbruck, T. Design of A Spatiotemporal Correlation Filter for Event-based Sensors. In Proceedings of the 2015 IEEE International Symposium on Circuits and Systems, Lisbon, Portugal, 24–27 May 2015. [CrossRef]
13. Khodamoradi, A.; Kastner, R. O(N)-Space Spatiotemporal Filter for Reducing Noise in Neuromorphic Vision Sensors. *IEEE Trans. Emerg. Top. Comput.* **2018**, *9*, 15–23. [CrossRef]
14. Guo, S.; Delbruck, T. Low Cost and Latency Event Camera Background Activity Denoising. *IEEE Trans. Pattern Anal. Mach. Intell.* **2022**, *45*, 785–795. [CrossRef] [PubMed]
15. Rios-Navarro, A.; Guo, S.; Abarajithan, G.; Vijayakumar, K.; Linares-Barranco, A.; Aarrestad, T.; Kastner, R.; Delbruck, T. Within-Camera Multilayer Perceptron DVS Denoising. In Proceedings of the 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, Vancouver, BC, Canada, 17–24 June 2023. [CrossRef]
16. Guo, S.; Kang, Z.; Wang, L.; Li, S.; Xu, W. HashHeat: An O(C) Complexity Hashing-based Filter for Dynamic Vision Sensor. In Proceedings of the 25th Asia and South Pacific Design Automation Conference, Beijing, China, 13–16 January 2020. [CrossRef]

17. Kumar, G.P.; Chang, C.H.; Basu, A. HAST: A Hardware-Efficient Spatio-Temporal Correlation Near-Sensor Noise Filter for Dynamic Vision Sensors. *IEEE Trans. Circuits Syst. I* **2025**, *72*, 1332–1345. [\[CrossRef\]](#)
18. Zhao, Q.; Ji, Y.; Wang, J.; Wu, J.; Shi, G. An O (m + n)-Space Spatiotemporal Denoising Filter with Cache-Like Memories for Dynamic Vision Sensors. *IEEE ACM Int. Conf. Comput.-Aided Des.* **2024**, *1*, 231. [\[CrossRef\]](#)
19. Zhao, Q.; Ji, Y.; Wang, J.; Wu, J.; Shi, G. Simultaneous Denoising and Compression for DVS with Partitioned Cache-Like Spatiotemporal Filter. In Proceedings of the 2025 Design, Automation and Test in Europe Conference (DATE), Lyon, France, 31 March–1 April 2025. [\[CrossRef\]](#)
20. Tian, H.; Fowler, B.; Gamal, A. Analysis of Temporal Noise in CMOS Photodiode Active Pixel Sensor. *IEEE J. Solid-State Circuits* **2001**, *36*, 92–101. [\[CrossRef\]](#)
21. Tian, H. Noise Analysis in CMOS Image Sensors. Doctoral Dissertation, Stanford University, San Francisco, CA, USA, 2000. Available online: https://isl.stanford.edu/abbas/group/papers_and_pub/hui_thesis.pdf (accessed on 23 June 2023).
22. Yadid, P.; Mansoorian, B.; Fossum, E. Optimization of Noise and Responsivity in CMOS Active Pixel Sensors for Detection of Ultralow-Light Levels. In Proceedings of the SPIE-the International Society for Optical Engineering 1997, San Diego, CA, USA, 29–31 July 1997. [\[CrossRef\]](#)
23. Mendis, S.K.; Kemeny, S.E.; Gee, R.C.; Pain, B.; Staller, C.O.; Fossum, E.R. CMOS Active Pixel Image Sensors for Highly Integrated Imaging Systems. *IEEE J. Solid-State Circuits* **1997**, *32*, 187–197. [\[CrossRef\]](#)
24. Decker, S.; Mcgrath, R.; Brehmer, K.; Sodini, C. A 256×256 CMOS Imaging Array with Wide Dynamic Range Pixels and Column-Parallel Digital Output. *IEEE J. Solid-State Circuits* **1998**, *33*, 2081–2091. [\[CrossRef\]](#)
25. Zhang, Y.X. Research on Feature Extraction and Classification Methods for DVS Event Streams. Master’s Thesis, Xidian University, Xi’an, China, 2019. [\[CrossRef\]](#)
26. Nozaki, Y.; Delbruck, T. Temperature and Parasitic Photocurrent Effects in Dynamic Vision Sensors. *IEEE Trans. Electron Devices* **2017**, *64*, 3239–3245. [\[CrossRef\]](#)
27. Yang, M.; Liu, S.; Delbruck, T. A Dynamic Vision Sensor with 1% Temporal Contrast Sensitivity and In-Pixel Asynchronous Delta Modulator for Event Encoding. *IEEE J. Solid-State Circuits* **2015**, *50*, 2149–2160. [\[CrossRef\]](#)
28. Li, J.; Tian, Y.H. Research Progress and Application Overview of Neuromorphic Vision Sensors. *Chin. J. Comput.* **2021**, *44*, 1258–1286. [\[CrossRef\]](#)
29. Feng, Y. Research on Event Stream Processing Methods for Dynamic Vision Sensors. Doctoral Dissertation, University of Chinese Academy of Sciences, Beijing, China, 2021. [\[CrossRef\]](#)
30. Lee, J.H.; Delbruck, T.; Pfeiffer, M.; Park, K.J.; Shin, C.W. Real-Time Gesture Interface Based on Event-Driven Processing from Stereo Silicon Retinas. *IEEE Trans. Neural Netw. Learn. Syst.* **2014**, *25*, 2250–2263. [\[CrossRef\]](#) [\[PubMed\]](#)
31. Weikersdorfer, D.; Adrian, D.B.; Cremers, D.; Conradt, J. Event-based 3D SLAM with a Depth-augmented Dynamic Vision Sensor. In Proceedings of the 2014 IEEE International Conference on Robotics and Automation (ICRA), Hong Kong, China, 31 May–5 June 2014. [\[CrossRef\]](#)
32. Bardow, P.; Davison, A.J.; Leutenegger, S. Simultaneous Optical Flow and Intensity Estimation from an Event Camera. In Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, USA, 27–30 June 2016. [\[CrossRef\]](#)
33. Vidal, A.R.; Rebecq, H.; Horstschaefer, T.; Scaramuzza, D. Ultimate SLAM? Combining Events, Images, and IMU for Robust Visual SLAM in HDR and High-Speed Scenarios. *IEEE Robot Autom. Lett.* **2018**, *3*, 994–1001. [\[CrossRef\]](#)
34. Czech, D.; Orchard, G. Evaluating Noise Filtering for Event-Based Asynchronous Change Detection Image Sensors. *IEEE Press* **2016**, *10*, 19–24. [\[CrossRef\]](#)
35. Ding, S.; Chen, J.; Wang, Y.; Kang, Y.; Song, W.; Cheng, J.; Cao, Y. E-MLB: Multilevel Benchmark for Event-Based Camera Denoising. *IEEE Trans. Multimed.* **2024**, *10*, 65–76. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.